# A Low Cost FPGA Implementation of Retinex Based Low-Light Image Enhancement Algorithm

Bharat Bhushan Upadhyay, *Graduate Student Member, IEEE*, and Kishor Sarawadekar, *Senior Member, IEEE*

***Abstract*—Real-time low-light image enhancement has several potential applications, such as advanced driver assistance systems (ADAS), remote sensing, object tracking, etc. The Retinex-based algorithms are mostly used to restore the visibility of low-light images. However, they perform complex mathematical operations over a large spatial window. Consequently, their hardware realization is tedious, and few researchers have attempted to address this problem. In this brief, we propose a Retinex-based algorithm that employs a low-cost edge-preserving filter for illumination estimation. Although certain approximations are used to curtail the hardware logic resource requirement, the quality of the enhanced image is not compromised. The proposed architecture requires only 10868 LUTs and 7409 registers when implemented on ZynQ 7 FPGA. Moreover, it can process HD images (1920×1080) at the rate of 60 frames per second (fps).**

***Index Terms*—Retinex algorithm, low-light image enhancement (LLIE), edge-preserving filter, FPGA implementation.**

## I. Introduction

LOW-LIGHT images suffer from poor visibility and color degradation. A real-time computer vision system is expected to work equally well under different illumination conditions. But low lighting conditions can severely affect its performance, as some information may be lost during the image acquisition [1], [2]. This problem can be mitigated by integrating an efficient low-light image enhancement (LLIE) subsystem. However, real-time implementation of a reliable and cost-effective LLIE system is intricate.

A concise review of LLIE methods is presented in [3]. Traditionally, histogram equalization (HE) methods were used to adjust the dynamic range of the low-light image. However, these methods achieve limited enhancement as they neglect the overall features of the image. Retinex-based low-light image enhancement methods have gained more popularity than HE methods. Retinex techniques decompose a low-light image into reflectance and illumination channels. These channels are further enhanced and recombined to obtain a visually pleasing image. In [4] and [5], the illumination channel is obtained using a Gaussian filter. An efficient LLIE technique, LIME, is presented in [6], which imposes some structural information to refine the initial illumination channel using the Augmented Lagrangian Multiplier (ALM). The refined illumination is further gamma-corrected to obtain the enhanced image. Since Retinex-based enhancement algorithms are complex, a fast and efficient method with reduced computational cost is proposed in [7]. Based on Retinex, an edge-preserving filter for illumination estimation is presented in [8]. It limits the range of illumination and preserves details in the reflectance.

In recent years, machine-learning-based LLIE models have flourished. A Retinex model-based enhancement network is introduced in [9]. It uses a separate network for reflectance decomposition and illumination enhancement. This end-to-end model performs very well illumination adjustment. Further, it enhances reflectance by denoising it. A highly effective GAN-based enhancement network proposed in [10] employs a global-local discriminator to handle the lighting conditions adaptively. It imposes a self-regularized attention model using the illumination map and eliminates the need for paired training data. Another Retinex-inspired, lightweight enhancement network is presented in [11], which simultaneously searches for an efficient architecture within a compact search space to estimate illumination and remove noise without paired or unpaired supervision. It uses a bilevel search strategy and distillation technique to reduce the processing time. An end-to-end low-light image enhancement model proposed in [12] uses illumination as a constraint to produce visually enhanced images. It uses feature extraction and channel attention mechanisms to adjust the light levels of poorly illuminated areas and restore image details.

Some practical LLIE applications, such as ADAS in modern vehicles for passenger safety, remote sensing using satellites for hazard assessment, etc., require low-power and cost-effective systems. Most of the effective LLIE algorithms are compute-intensive and require large memory, especially machine learning-based algorithms. Methods using high-performance GPUs like [9], [10], [11], and [12], become not only power hungry but also costly. This restricts their suitability for real-time implementation on reconfigurable devices with limited resources. Researchers have tried to implement Retinex-based LLIE algorithms on FPGAs in real-time with several modifications to the base algorithm. FPGA implementation of multi-scale Retinex-based LLIE is proposed in [13], which replaces Gaussian filtering with blurring and substitutes logarithmic values with pre-calculated values. Still, it requires a large number of hardware resources. The algorithm proposed in [7] is modified to attain real-time implementation on the FPGA platform in [14]. Here, the hardware cost is reduced

Manuscript received 9 January 2024; accepted 30 January 2024. Date of publication 2 February 2024; date of current version 1 July 2024. This brief was recommended by Associate Editor Z. Yu. *(Corresponding author: Kishor Sarawadekar.)*

The authors are with the Department of Electronics Engineering, Indian Institute of Technology (BHU) Varanasi, Varanasi 221005, India (e-mail: skishor.ece@iitbhu.ac.in).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TCSII.2024.3361561.

Digital Object Identifier 10.1109/TCSII.2024.3361561

1549-7747 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on February 20,2026 at 11:39:42 UTC from IEEE Xplore. Restrictions apply.

by employing approximate computations and replacing frame buffers with line buffers. However, some details are lost in the decomposed reflectance. A real-time implementation of a tone mapping algorithm with halos suppression capability is presented in [15]. An adaptive histogram equalization-based FPGA implementation of low-light video enhancement is proposed in [16], which also suppresses haze.

LLIE algorithms involve complex mathematical operations. Therefore, the hardware resource requirements of the aforementioned FPGA implementations are very large. To address this issue, we present a Retinex-based LLIE algorithm in this brief, which uses the concept of approximate computing to reduce hardware cost. Retinex-based low-light scene enhancement methods efficiently restore images with balanced color and preserved edges. The key contributions of this brief are:

- A low-complexity edge-preserving filter is proposed for illumination estimation, which aids in retaining fine details in the decomposed reflectance.
- An approximate computing method is used to implement exponentiation operation, thereby reducing hardware cost.
- FPGA implementation of the proposed algorithm can process 60 HD (1920×1080) frames per second.

## II. The Retinex Model

According to the Retinex model [4], an image $P$ is given as the product of illumination $E$ and reflectance $R$ as

$$P(x, y) = E(x, y) \times R(x, y) \tag{1}$$

where x and y are the pixel coordinates. The illumination channel $E$ represents the light intensity. The Retinex model assumes that $E$ should be devoid of textures and vary smoothly. On the contrary, the reflectance channel $R$ comprises textures and details. Estimation of $E$ using the Retinex model should retain edge information and smooth out textural details in it.

## III. The Proposed Enhancement Algorithm

For an input image $P$, we obtain the coarse illumination $P^{ci}$ by finding the maximum intensity in a local patch $\Delta$ centred at coordinate (x,y) as shown below

$$P^{ci}(x, y) = \max_{(x,y)\in\Delta} \{ \max_{c\in(R,G,B)} P^c(x, y)\} \tag{2}$$

where c is the R, G, B color channel of $P$. However, the coarse illumination estimated using (2) cannot be used for reflectance estimation as it contains block effects.

An edge-preserving filter is proposed in [8] for efficient illumination estimation. It preserves sharp edges in the illumination channel and maintains spatial smoothness. This filter performs edge preservation based on guidance image $G_i$, which is also the maximum channel of $P$, and it is given as

$$G_i(x, y) = \max_{c\in(R,G,B)} \left(P^c(x, y)\right). \tag{3}$$

$G_i$ retains the edges and imposes a theoretical lower bound on illumination. The filtered output $F$, which is also the refined illumination, is given as

$$F(x, y) = \frac{\sum_{(m,n)\in\Omega} \beta(m, n)\gamma(m, n)P^{ci}(m, n)}{\sum_{(m,n)\in\Omega} \beta(m, n)\gamma(m, n)} \tag{4}$$

where $\Omega$ is a local window centered around a pixel in $P^{ci}$. Here, $\beta$ and $\gamma$ are defined as

$$\beta(m, n) = \begin{cases} 1, & P^{ci}(m, n) - G_i(x, y) \geq 0 \\ 0, & \text{otherwise} \end{cases}, \tag{5}$$

$$\gamma(m, n) = exp\left(-\frac{\left|P^{ci}(m, n) - G_i(x, y)\right|}{2\sigma^2}\right) \tag{6}$$

where $\sigma$ controls the range similarity. The filter obtains the refined illumination corresponding to a pixel in the guidance image $G_i$ by computing the weighted average of pixels in the coarse illumination $P^{ci}$ over a region $\Omega$.

The region $\Omega$ is an odd-size square patch or window. The averaging operation over the entire window would require many multipliers and adders. This increases the total hardware cost. However, to reduce the hardware complexity, we replaced each column of the 2-D window $\Omega$ with the minimum value of coarse illumination pixels occupying the respective column of $\Omega$. This transforms the 2-D filter into a 1-D filter of size 1×r, where r is the number of columns in the given window. The output of the proposed edge-preserving filter is used as the refined illumination $F_m(x, y)$, which is given as

$$F_m(x, y) = \frac{\sum_1^r \beta_m(k)\gamma_m(k)P^{ci}_{min}(k)}{\sum_1^r \beta_m(k)\gamma_m(k)} \tag{7}$$

where $P^{ci}_{min}(k)$ represents the minimum value of the $k^{th}$ column of the 2-D window centered at location (x,y) of coarse illumination $P^{ci}(x, y)$. Further, $\beta_m$ and $\gamma_m$ are given as

$$\beta_m(k) = \begin{cases} 1, & P^{ci}_{min}(k) - G_i(x, y) \geq 0 \\ 0, & \text{otherwise} \end{cases} \tag{8}$$

$$\gamma_m(k) = exp\left(-\frac{\left|P^{ci}_{min}(k) - G_i(x, y)\right|}{2\sigma^2}\right). \tag{9}$$

Once $F_m(x, y)$ is estimated, reflectance $R(x,y)$ can be obtained as given below

$$R^c(x, y) = \frac{P^c(x, y)}{F_m(x, y)} \tag{10}$$

where $c \in$ (R,G,B). The main advantages of this method are

- The proposed filter preserves edges and eliminates blocky effects in $F_m(x, y)$ by assigning lower weights to pixels that differ more from the guided image pixels, as can be observed from (9).
- The proposed filter limits the lower bound of $F_m(x, y)$ to the maximum channel using (8). This further limits the reflectance in the range [0, 1] and prevents an overflow of reflectance values, thereby preserving fine details.

$F_m(x, y)$ is further modified using the contrast stretch function given by (11) to obtain modified illumination $E_{mod}(x, y)$. The contrast stretch function employs the normalized pixel value of $F_m(x, y)$. Finally, $E_{mod}(x, y)$ and $R(x, y)$ are remapped using (12) to obtain the visually enhanced image $P_E$. These operations are mathematically represented as

$$E_{mod}(x, y) = \sqrt{F_m(x, y)}\sqrt{2 - F_m(x, y)}, \tag{11}$$

$$P_E(x, y) = E_{mod}(x, y) \times R(x, y). \tag{12}$$

The quality of the output image and the hardware resource requirement depend on the size of windows $\Delta$ and $\Omega$. Both

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on February 20,2026 at 11:39:42 UTC from IEEE Xplore. Restrictions apply.



(a) (b) 3×3 (c) 11×11 (d) 31×31

Fig. 1. Illumination obtained with different sizes of local window. (a) Low-light image. (b)-(d) Illumination with different window sizes.

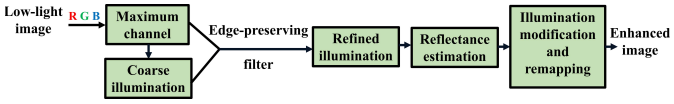



Fig. 2. Block diagram of the proposed architecture.

the windows are identical in shape and size. But their region of operation is different. Please note that Δ and Ω operate on $G_i$ and $P^{ci}$, respectively. To determine an appropriate window size, we experimented with various window sizes and tested the proposed method on several low-light images. Fig. 1 shows illumination obtained using different local window sizes. Illumination obtained with a smaller window (3×3) size contains a lot of texture, whereas edge information is lost with a larger window (31×31) size. Fig. 1 depicts this fact. The illumination obtained with the 11×11 window size is devoid of textures, and edge information is retained properly. Moreover, the hardware resource requirement for 11×11 window size is also moderate. Therefore, we chose 11×11 window size in this brief. Another important aspect of the proposed method is boundary handling, which is dealt with by padding the image with stripes of width (r-1)/2, where r is the size of the local window. Further, these stripes are filled with mirror images of boundary elements in the image.

## IV. Architecture of the Proposed Algorithm

The block diagram of the proposed enhancement architecture is shown in Fig. 2. First, the maximum channel $G_i$ of the input image $P$ is obtained by finding the maximum intensity in all three color channels. The input pixels, along with $G_i$, are stored in line buffers. Once the $11^{th}$ row of $G_i$ is available in the line buffers, a 11×11 size window Δ is centered around each pixel in $G_i$. The maximum value of each column in Δ is obtained as shown in Fig. 3(a). Next, the obtained column maximums are buffered in a register set. Finally, the maximum value of the register set gives the window maximum, i.e., coarse illumination pixel $P^{ci}$, as shown in Fig. 3(b). As the local window slides one pixel to the right, it can be observed from Fig. 3(c) that the two consecutive windows differ by only one column. Thus, to find the maximum value within the new window, we need to find the maximum value within the new column and write it in the register set, which holds the maximum value for each column. The newly added column's maximum value is written in place of the excluded column's maximum value, and the maximum

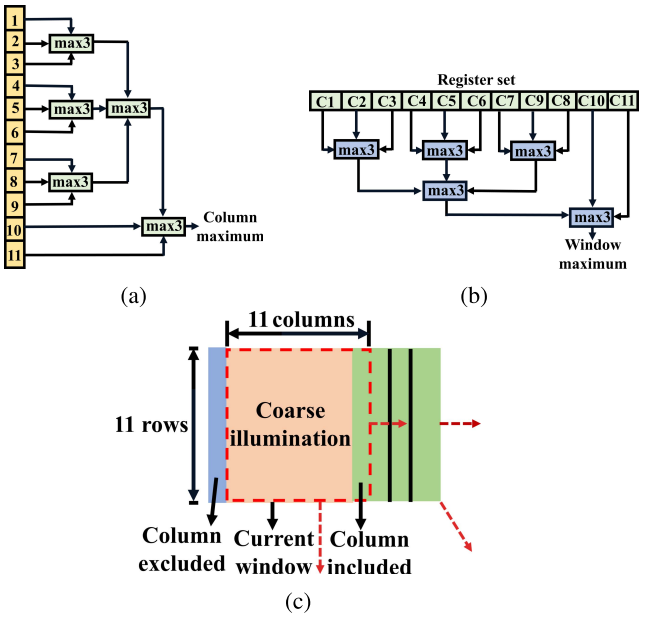
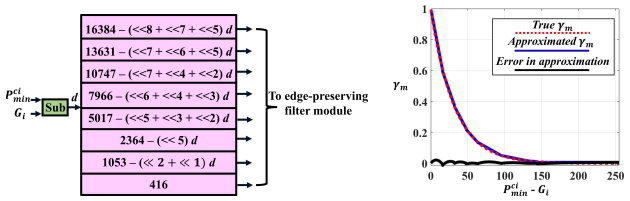



Fig. 3. Architecture of the local maximum module. (a) Searching the maximum value in a column. (b) Searching the maximum value in a window (c) Sliding window structure.

value in the register set gives the maximum value for the newly formed local window. The local window Δ slides in raster scan format over the entire maximum channel $G_i$, and each local window centered at (x,y) gives the coarse illumination pixel $P^{ci}$ at (x,y), which are also stored in line buffers.

Once sufficient lines of $P^{ci}$ are available, a sliding window Ω is centered around each pixel of $P^{ci}$. The minimum value of the $k^{th}$ column in Ω, i.e., $P^{ci}_{min}(k)$ is computed using the architecture shown in Fig. 3(a), replacing the max operation with the min operation. Each $P^{ci}_{min}(k)$ corresponding to a specific position of Ω is stored in a new register set. Next, the difference $d$ between $P^{ci}_{min}(k)$ and $G_i$ is calculated and fed to the linear approximation module to obtain $\gamma_m(k)$ using (9). We have set $\sigma$ to 4 in this brief. To precisely calculate $\gamma_m(k)$ with reduced computational complexity, the $\gamma_m(k)$ function is divided into eight linear segments based on the range of $d$. Each segment is represented by a straight-line equation in slope-intercept form. Each input to the exponential function is assigned a distinct output value by multiplying the input with the slope of the linear segment and adding the intercept value to it. The slopes of each of the eight linear segments are set so that their multiplication can be achieved by, at most, three distinct left shifts and two addition operations. The linear approximation module outputs eight values of $\gamma_m(k)$ in all the eight linear regions for each $d$. However, depending on the value of $d$, only one of the eight $\gamma_m(k)$s is useful. The architecture of the linear approximation module is shown in Fig. 4(a), which also depicts the slope and intercept values of each linear segment. The slope and intercept have been scaled by $2^{14}$ to reduce approximation loss. The range of each linear segment is presented in Fig. 4(b). The entire linear approximation module is implemented using shifters and adders. In this way, we have eliminated the requirement for multipliers. A similar technique is used in [17] for exponential

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on February 20,2026 at 11:39:42 UTC from IEEE Xplore. Restrictions apply.




(a) (b)

Fig. 4. (a) Architecture of the linear approximation module. (b) Plot of $\gamma_m(k)$.

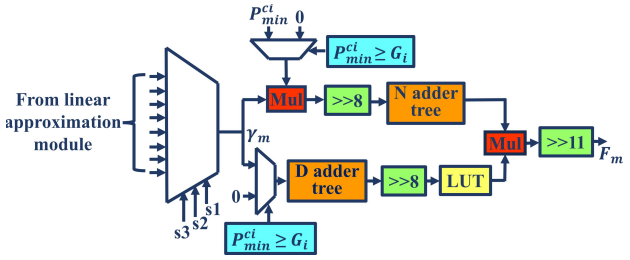
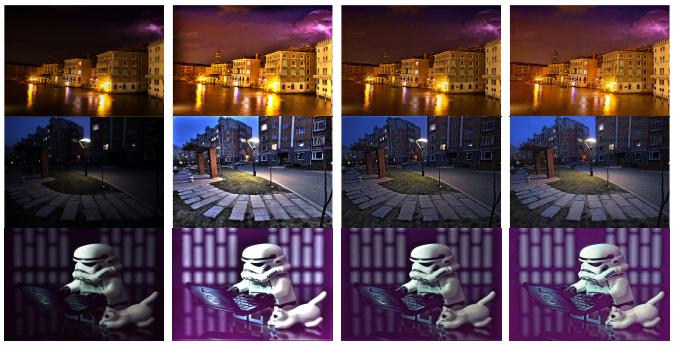



Fig. 5. Architecture of the edge-preserving filter module.

approximation. However, the exponential function in [17] is divided into several intervals, and pre-computed output values for each interval are stored in LUT-based memory. Depending on the interval in which the input falls, a particular value is sent to the output. It can be observed from Fig. 4(b) that the maximum error in $\gamma_m(k)$ is not more than 0.03, even if we adopt the proposed approximation technique. The multiplexer at the input of the edge-preserving filter module, shown in Fig. 5, selects an appropriate $\gamma_m(k)$ value depending on the range of $d$. Next, according to the condition $P^{ci}_{min}(k) \geq G_i$ the weights in the numerator and denominator for each coarse illumination pixel $P^{ci}_{min}(k)$ is calculated using (7) and (8). It is clear from (7) that the sum of the weights in the numerator (N) and denominator (D) are different. So, two N and D adder trees are used in this module, as shown in Fig. 5. Finally, the output of the N adder tree is divided by the output of the D adder tree to obtain the refined illumination pixels $F_m(x, y)$. Here, the division operation is replaced with a multiplier and a lookup table to reduce the hardware resource requirement and improve the latency. Next, based on (10), $R(x, y)$ is calculated in all three color channels using $P(x, y)$ and $F_m(x, y)$ obtained from previous modules. To simplify the hardware, the complex divider in (10) is replaced with a multiplier and a lookup table that stores the probable values of $1/F_m(x, y)$. Next, $F_m(x, y)$ is modified as given in (11). To reduce hardware complexity, the nonlinear modification in $F_m(x, y)$ is attained using a LUT-based memory that stores the pre-computed modified values $E_{mod}(x, y)$ stored at the address specified by $F_m(x, y)$. Finally, $E_{mod}(x, y)$ is remapped with $R(x, y)$ to produce an enhanced output image using (12). The entire architecture is pipelined so that a throughput of one pixel per clock cycle can be achieved.

## V. Results and Discussion

Experimental results obtained using the proposed method are discussed in this section. Qualitative results obtained using the proposed method are shown in Fig. 6. We used the



(a) (b) (c) (d)

Fig. 6. Visual results (a) Low-light image. (b) Results with [6]. (c) Results with [14]. (d) The proposed method.



(a) (b) (c)

Fig. 7. Reflectance obtained with different methods. (a) Low-light image. (b) Reflectance with [14]. (c) Reflectance with the proposed method.

TABLE I
Comparison With State-of-the-Art Methods Using PSNR and SSIM Metrics on the LOL Dataset

| | [6] | [9] | [10] | [11] | [12] | The proposed |
|---|---|---|---|---|---|---|
| **PSNR↑** | 17.18 | **17.607** | 17.551 | 16.532 | 15.107 | 17.021 |
| **SSIM↑** | 0.562 | 0.654 | 0.665 | 0.527 | 0.564 | **0.7511** |

methods of [6] and [14] for visual comparison, as those are also Retinex-based methods. It can be observed from Fig. 6 that images obtained using the proposed method are neither over-enhanced nor under-enhanced like others. Moreover, it is clear from Fig. 7 that the proposed method effectively limits the range of reflectance in [0, 1] as compared to [14]. We used the LOL [9] dataset for quantitative evaluation of the proposed method using reference metrics peak signal-to-noise ratio (PSNR) [18] and structural similarity (SSIM) [18]. Higher PSNR and SSIM values signify better restoration and a higher resemblance of the recovered image to the ground truth. We also used DICM [12] and LIME [6] datasets for testing the performance of the proposed algorithm using the no-reference metrics natural image quality evaluator NIQE [12] and blind/referenceless image spatial quality evaluator (BRISQUE) [19]. A lower value of NIQE and BRISQUE signifies more naturalness in the image. We compared the proposed method with other state-of-the-art methods using these metrics, and the results are presented in Tables I and II. It is clear from Tables I and II that the best PSNR score is obtained with [9], whereas the best SSIM score is obtained with the proposed method. However, the PSNR score of the proposed method is comparable to the other methods. Regarding the BRISQUE score, the proposed method outperforms the other methods on the DICM dataset, and it has the second-best score for the LIME dataset. The

Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on February 20,2026 at 11:39:42 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON WITH STATE-OF-THE-ART METHODS USING NIQE (NIQ) AND BRISQUE (BRI) METRICS ON DICM AND LIME DATASETS

| Methods | Datasets | | | |
|---|---|---|---|---|
| | DICM | | LIME | |
| | NIQ↓ | BRI↓ | NIQ↓ | BRI↓ |
| **[6]** | 4.57 | 25.25 | 3.85 | **21.53** |
| **[9]** | 4.19 | 30.05 | 4.82 | 27.03 |
| **[10]** | 3.56 | 29.31 | 3.73 | 23.77 |
| **[11]** | 4.55 | 36.63 | 3.66 | 29.96 |
| **[12]** | **3.40** | NA | **3.65** | NA |
| **The proposed** | 3.48 | **24.64** | 3.67 | 22.41 |

TABLE III
COMPARISON OF HARDWARE RESOURCE UTILIZATION

| Architectures | [13] | [14] | [15] | [16] | The proposed |
|---|---|---|---|---|---|
| LUTs | 20043 | 24176 | 93989 | 28362 | 10868 |
| Registers | 420331 | 22787 | - | 7795 | 7409 |
| SRAM | 9252Kb | 3348Kb | 87Kb | 358Kb | 1152Kb |
| DSP48Es | - | 13 | 28 | 105 | 22 |
| Frequency (MHz) | 200 | 148.5 | 100 | 75.84 | 148.5 |
| Resolution | 640×480 | 1920×1080 | 1240×768 | 1920×1080 | 1920×1080 |
| Frame rate (fps) | 60 | 60 | 126 | 30 | 60 |

NIQE score of the proposed method is much better than some existing methods.

The hardware architecture of the proposed method has been designed in the AMD-Xilinx Vivado design suite. We used Verilog for implementation and AMD-Xilinx IPs for video interfacing. The proposed architecture is synthesized on a ZynQ 7 series FPGA, and its resource utilization is shown in Table III. It is evident from this table that the proposed architecture requires only 45% of LUTs as compared to [13]. Further, the register count of the proposed design is also lower than the rest of the designs. Design [13] operates at a high frequency, but it supports the lowest frame resolution. Additionally, its hardware resource requirement is very high because large-size box filters are used to perform Gaussian blurring. Design [15] achieves the highest frame rate but at a lower resolution. Moreover, a halo-reducing filter is essential in this method, which increases its hardware cost. The same image resolution is supported in designs [14], [16], and ours. Architecture [14] requires the least number of DSP resources. However, its LUT and register consumption are higher due to the large Gaussian filter used in illumination estimation. Since the operating speed of [16] is less, it fails to support a high frame rate. We have set the operating frequency of the proposed architecture to 148.5 MHz, at which AMD-Xilinx video processing IPs operate. This operating frequency is sufficient to process full HD (1920×1080) images at 60 fps.

## VI. CONCLUSION

In this brief, we present the real-time implementation of a Retinex-based low-light image enhancement algorithm on a ZynQ FPGA. The proposed algorithm produces high-quality output images by employing a low-complexity edge-preserving filter for illumination refinement. The hardware requirement of the proposed architecture is significantly less than the existing methods, and it can process 60 images of full HD resolution in a second. This proves the candidacy of the proposed method for real-time applications such as ADAS, remote sensing, object detection, etc.

## REFERENCES


[1] H. Wang et al., "SFNet-N: An improved SFNet algorithm for semantic segmentation of low-light autonomous driving road scenes," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21405–21417, Nov. 2022.

[2] E. Khatab, A. Onsy, M. Varley, and A. Abouelfarag, "Vulnerable objects detection for autonomous driving: A review," *Integration*, vol. 78, pp. 36–48, May 2021.

[3] W. Wang, X. Wu, X. Yuan, and Z. Gao, "An experiment-based review of low-light image enhancement methods," *IEEE Access*, vol. 8, pp. 87884–87917, 2020.

[4] D. J. Jobson, Z. Rahman, and G. A. Woodell, "Properties and performance of a center/surround retinex," *IEEE Trans. Image Process.*, vol. 6, pp. 451–462, 1997.

[5] D. J. Jobson, Z. Rahman, and G. A. Woodell, "A multiscale retinex for bridging the gap between color images and the human observation of scenes," *IEEE Trans. Image Process.*, vol. 6, pp. 965–976, 1997.

[6] X. Guo, Y. Li, and H. Ling, "LIME: Low-light image enhancement via illumination map estimation," *IEEE Trans. Image Process.*, vol. 26, pp. 982–993, 2017.

[7] Y. Shin, S. Jeong, and S. Lee, "Efficient naturalness restoration for non-uniform illumination images," *IET Image Process.*, vol. 9, no. 8, pp. 662–671, 2015.

[8] Y. Gao, H.-M. Hu, B. Li, and Q. Guo, "Naturalness preserved nonuniform illumination estimation for image enhancement based on retinex," *IEEE Trans. Multimedia*, vol. 20, no. 2, pp. 335–344, Feb. 2018.

[9] C. Wei, W. Wang, W. Yang, and J. Liu, "Deep retinex decomposition for low-light enhancement," in *Proc. Brit. Mach. Vis. Conf.*, 2018, pp. 1–12.

[10] Y. Jiang et al., "EnlightenGAN: Deep light enhancement without paired supervision," *IEEE Trans. Image Process.*, vol. 30, pp. 2340–2349, 2021.

[11] R. Liu, L. Ma, J. Zhang, X. Fan, and Z. Luo, "Retinex-inspired unrolling with cooperative prior architecture search for low-light image enhancement," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 10561–10570.

[12] G.-D. Fan, B. Fan, M. Gan, G.-Y. Chen, and C. L. P. Chen, "Multiscale low-light image enhancement network with illumination constraint," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 11, pp. 7403–7417, Nov. 2022.

[13] D. I. Ustukov, Y. R. Muratov, and V. N. Lantsov, "Modification of retinex algorithm and its stream implementation on FPGA," in *Proc. 6th Mediterr. Conf. Embed. Comput. (MECO)*, 2017, pp. 1–4.

[14] J. W. Park et al., "A low-cost and high-throughput FPGA implementation of the retinex algorithm for real-time video enhancement," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 1, pp. 101–114, Jan. 2020.

[15] P. Ambalathankandy, A. Horé, and O. Yadid-Pecht, "An FPGA implementation of a tone mapping algorithm with a halo-reducing filter," *J. Real-Time Image Process.*, vol. 16, no. 4, pp. 1317–1333, Aug 2019.

[16] C. Xu, Z. Peng, X. Hu, W. Zhang, L. Chen, and F. An, "FPGA-based low-visibility enhancement accelerator for video sequence by adaptive histogram equalization with dynamic clip-threshold," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 11, pp. 3954–3964, Nov. 2020.

[17] F. Spagnolo, P. Corsonello, F. Frustaci, and S. Perri, "Design of approximate bilateral filters for image denoising on FPGAs," *IEEE Access*, vol. 11, pp. 1990–2000, 2023.

[18] S. K. Yadav and K. Sarawadekar, "Effective edge-aware weighting filter-based structural patch decomposition multi-exposure image fusion for single image dehazing," *Multidimens. Syst. Signal Process.*, vol. 34, no. 2, pp. 543–574, Jun. 2023.

[19] A. Mittal, A. K. Moorthy, and A. C. Bovik, "No-reference image quality assessment in the spatial domain," *IEEE Trans. Image Process.*, vol. 21, pp. 4695–4708, 2012.


Authorized licensed use limited to: Thapar Institute of Engineering & Technology. Downloaded on February 20,2026 at 11:39:42 UTC from IEEE Xplore. Restrictions apply.